Reliable agent workflows

Routing, workflow reuse and privacy evaluation with Phoenix

When can an agent spend less, reuse prior work, or transform its traces without hiding mistakes? This Python project studies those questions through three separate controlled experiments and an observable execution path.

-39.7%
Cost per mechanical success

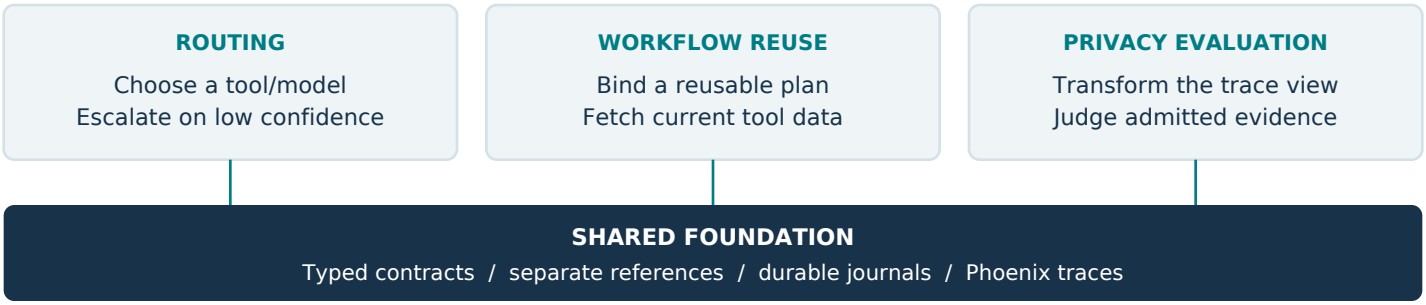
-45.7%
Median endpoint latency

122/128
Exact and fresh successes

Headline: exact workflow reuse versus fresh planning on the completed held-out study. Success includes 100 completed responses and 22 appropriate clarifications in each condition; six abstentions remain unresolved.

One project, three explicit questions

Study	Question	Evidence status
Workflow reuse	Can recurring requests skip planning while fetching current facts?	**Measured:** 768 held-out endpoints; exact reuse is the supported v1 choice.
Privacy evaluation	After identifiers change, can a judge still detect the wrong resource?	**Development observations:** 192 real model judgments; human and held-out judgments pending.
Confidence routing	Does uncertainty identify mistakes a larger model can repair?	**Implemented:** 120 authored examples and synthetic mechanics; measured conclusions pending.



These studies share engineering controls, but use separate datasets and protocols. Real model calls in the reuse and privacy studies operate on controlled synthetic fixtures; this is not production traffic.

What is worth presenting

The strongest result is a boundary: valid, authorized tool arguments can still implement the wrong intent. Exact reuse captured the measured efficiency gain with simpler behavior. Privacy evaluation shows why zero false approvals must be read alongside evaluator coverage. Routing makes the same question testable with an uncertainty gate and an escalation control.

Reading map: 2 / reuse and its failure case | 3 / privacy and abstention | 4 / routing, engineering and next steps

01 / COMPLETED MEASURED STUDY

Reuse the workflow. Fetch the current facts.

A frozen bank stores resource-free plans: typed slots, ordered steps, dependencies and provenance. A binder inserts current identifiers and operands. Accepted plans still call current loopback HTTP tools and the common answer model; misses invoke the same structured planner.

Warm policy	Mechanical successes	USD / success	Median latency
Fresh fused planning	122 / 128	$0.001853	3,490 ms
Exact reuse	122 / 128	$0.001117	1,896 ms
Verified semantic reuse	122 / 128	$0.001141	1,922 ms
Ordinary semantic reuse	106 / 128	$0.001028	1,776 ms

Held-out design: 64 base clusters x 6 conditions x 2 repetitions = 768 endpoints. The table shows four warm policies; fresh and verified empty-bank controls complete the six conditions. Development calibrated retrieval before the bank and settings were frozen.

A valid tool call can still be the wrong workflow

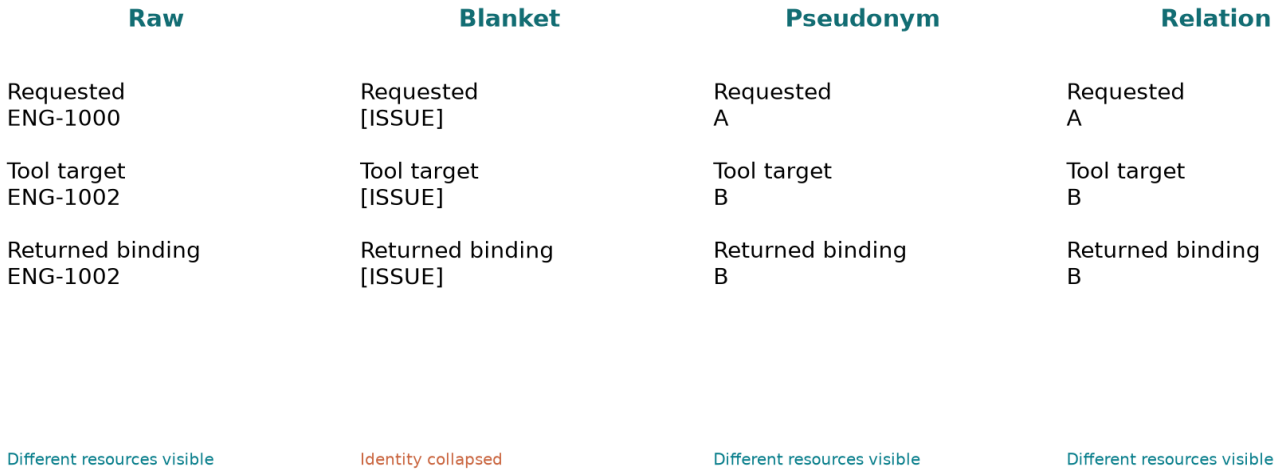
Saved subtraction request: calculate 4 minus 10408	Observed execution and outcome
Ordinary semantic reuse	Accepted an addition plan: **add(4, 10408) = 10412**. The final response noticed the mismatch but did not resolve the subtraction request.
Verified comparator	Rejected the semantic mismatch, made a fresh planning call, and executed **subtract(4, 10408) = -10404**.

Ordinary reuse accepted 18 incorrect paths among 94 hits. No observed authorization violation did not establish intent correctness. The measured Phoenix gallery exposes the request, model spans, actual tool arguments/results and mechanical grades. A paired issue lookup shows one LLM call under exact reuse versus two under fresh planning.

Recommendation: use exact reuse for v1 on this workload. Verified reuse admitted four additional hits without improving the efficiency headline. Miss-path tails, provider token caching, family wording and the small recurring workload limit transfer. Human answer review: 24/24 candidate-confirmed diagnostic judgments; original scores unchanged.

Preserve the evidence a judge needs.

A final answer can sound plausible even when a tool accessed the wrong issue. This study fixes the execution and answer, changes only the evaluator's view, and asks whether resource scope can still be judged. It compares raw text, blanket placeholders, trace-local pseudonyms and a relation view.



One saved wrong-resource execution requests ENG-1000 but targets and returns ENG-1002. A/B are display aliases for opaque tokens. Blanket replacement collapses distinct resources to the same string; matching placeholders cannot establish matching identity.

Coverage changes the meaning of a zero-error count

View	Correct / decided	Coverage	Abstentions	False approvals
Raw	48 / 48	100%	0	0 / 24
Blanket	0 / 0 (undefined)	0%	48	0 / 24
Pseudonym	48 / 48	100%	0	0 / 24
Relation	48 / 48	100%	0	0 / 24

Development scope: 12 base requests x 2 paired episodes x 4 views x 2 judgments = 192 real model judgments, or 48 per view. Both deterministic code baselines match the same pattern. The corpus has 72 captured canonical executions; the held-out judge phase is separate.

What this establishes - and what remains open

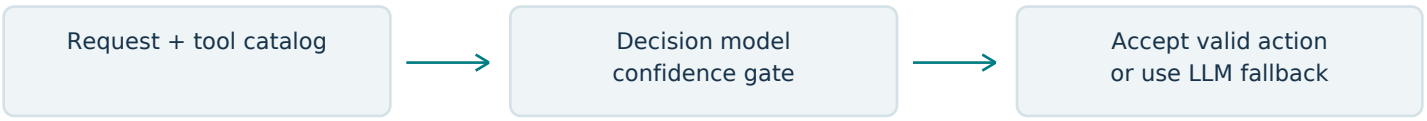
Identity-preserving views retained enough evidence for the admitted direct-lookup checks. Blanket masking avoided false approvals by abstaining on every judgment. That is information loss, not evidence of equally useful evaluation.

The evaluator relies on trusted local extraction of requested references, tool targets, authoritative returned bindings and observed links. This is an explicit observation contract, not a demonstrated solution to extracting identity or permissions from arbitrary traces.

Pending: human answerability audits and 384 held-out model judgments. These development observations prove neither model superiority nor a production privacy guarantee. Repeated judgments are paired within their base request; they are not independent opportunities to establish safety.

Escalate when uncertainty is useful.

The router chooses exactly one next action from eight: documentation, code, issue, deployment, stored test results, calculation, direct answer or clarification. Decisions-only, LLM-only and a hybrid confidence gate make the cost/correctness tradeoff inspectable.



Invalid output forces fallback; gate selection uses development data only.

Protocol: 120 authored examples, split into 40 development and 80 held-out requests. Related paraphrases stay in one split. A matched-rate random escalation control tests whether confidence adds value beyond buying the same number of extra calls.

The current routing numbers are explicitly synthetic

Offline mechanics demonstration	Accuracy	Interpretation
Decisions-only	62.5%	Eight synthetic examples, repeated three times per policy.
LLM-only	75.0%	A fixture replay for checking analysis mechanics.
Hybrid	75.0%	Six rescues and three regressions across 24 hybrid rows.

These are not real provider comparisons or results for the 120-example benchmark. Human label review, measured development/test routing runs and a measured recommendation remain pending.

Engineering lessons shared by all three studies

Trust boundaries: separate evaluator references from runtime inputs; validate typed outputs, tool identities and returned bindings.
Interruptions: budget and execution journals preserve ambiguous paid work and prevent automatic duplicate retries.
Measurement: retain failures in cost denominators, exclude later publication from response latency, and preserve paired request clusters.

A clean handoff and the next experiment

Application code, tests, the notebook, frozen evidence and four Phoenix captures remain in the repository. This is its only PDF; standalone helper and presentation scripts are removed.

```
uv sync --locked --group dev
uv run pytest -q
uv run ruff check src tests
uv run router-lab --help
```

Next: retain the completed 24-item diagnostic review alongside the original scores. Complete privacy and routing reviews before any new paid held-out runs. No new model calls were needed for this presentation.

Source of claims: frozen saved rows, independent references, original study verification and the 38-file notebook evidence bundle. Phoenix traces reconstruct saved measured executions; existing grades are mechanical CODE evaluations. See README, docs/status.md and the study guides for the full provenance.